Spade MVP — Testing & Local Setup Guide

How to run Spade locally and what to verify, end to end.

`tui-pilot · Spade MVP · generated 2026-06-15`

Spade turns product work into agent execution. The loop you are testing:

> **Create a Task** → **ground it in the Product Brain** (link features / decisions / feedback) → **run a 4-stage agent Pipeline** (Developer → Reviewer → Integrator → Documentor) on the project's cloud account → **watch it auto-advance and ship**. Backlog, Brain graph, Pipelines and Home give you the full picture.

1 · Prerequisites one-time

All already present on this machine — listed so you can reproduce elsewhere.

Requirement	Check	Notes
Python venv	`.venv/bin/python --version` → 3.12	Deps: fastapi, uvicorn, pyyaml (already installed).
tmux	`tmux -V` → 3.6a	Every agent runs in its own tmux session.
claude CLI	`which claude`	Needed only to actually *run* a pipeline (spawns real agents).
An authed account dir	`ls -d ~/.claude-*`	e.g. `~/.claude-t2b`. A logged-in `CLAUDE_CONFIG_DIR`. Required for pipeline execution.

2 · Start the server already running

A server is **already running for you** at `http://127.0.0.1:8765/ui/` using a clean, throwaway data home at `~/spade-qa`. Open that URL in your browser to begin.

To start it yourself next time (from the repo root):

```
# clean, disposable data dir (sqlite db + managed account dirs live here)
export TUI_PILOT_HOME=~/spade-qa

# launch the API + UI
.venv/bin/python -m uvicorn tui_pilot.server:app --host 127.0.0.1 --port 8765

# then open:  http://127.0.0.1:8765/ui/
```

One server is all you need. The single uvicorn process serves the web UI, the REST API, and spawns/drives every agent via tmux. There is no separate frontend/build step. `API docs` (top-right link) opens the live OpenAPI explorer.

3 · The test plan

Work top to bottom — each section sets up the next, ending in a real pipeline run. The nav rail is on the far left: **Home · Backlog · Brain · Pipelines · Fleet · Projects · Accounts · Agents · Settings**.

1 Accounts — connect a cloud account

a. Nav rail → **Accounts** (🔑). Click **Scan** — it should list your `~/.claude-*` dirs as importable candidates.

b. Import one you are logged into (e.g. **Time2Build** → `~/.claude-t2b`). Give it a colour.

c. Click **Set default** on it (the ⭐).

> **Expect:** a card with a coloured swatch, the `claude-code` provider tag, the config-dir path, and a **default** ⭐ badge. An un-authed dir shows *not logged in* with a **Log in** button (which opens a real login session you complete once).

2 Projects — create one and bind accounts

a. Nav → **Projects** (📦) → **+ New**. Name it (e.g. *Acme Storefront*), set the working dir to a **sandbox** path you don't mind an agent writing to — e.g. `/tmp/spade-demo` (create it first: `mkdir -p /tmp/spade-demo`).

b. Strategy **single**; add your account to the pool. (Try **round-robin** with two accounts to see the "next worker →" preview rotate.)

c. Make it the **current project** (project switcher, top bar). All other views are scoped to it.

> **Expect:** the top-bar chip shows the project name with coloured account dot(s); the pool reorders with the ↑↓ controls and the "next worker" preview updates.

> **Use a sandbox working dir.** A real pipeline's Developer/Integrator agents *edit files* in the project's working dir. Point it at a scratch folder for testing, not a repo you care about.

3 Agents — review the role library

a. Nav → **Agents** (🤖). You should see Planner, Developer, Reviewer, Integrator, Documentor, Plain, Orchestrator.

b. Open one and tweak its instructions / default model; save.

> **Expect:** edits persist; there is **no account field** (agents inherit the account from the project at spawn — that's the whole design).

4 Backlog — create & move tasks

a. Nav → **Backlog** (📋) → **+ Task**. Add 2–3 tasks with a feature name and priority (P0–P3).

b. Open a card; use the move buttons to push it Ready → In Progress → Review.

> **Expect:** each task gets a sequential id `SPD-001`, `SPD-002` … (workspace-wide). Cards show the feature chip and a priority dot (P0 red → P3). Moving a card re-columns it instantly; counts in the headers update.

5 Product Brain — build the graph & ground a task

a. Nav → **Brain** (🧠) → **+ Node**. Add a few typed nodes: a `feature` ("Checkout"), a `decision` ("ADR-7: Stripe"), a `feedback` ("Apple Pay requests").

b. + Edge between two nodes with a relation label (e.g. *decided_by*).

c. Back in **Backlog**, open a task → use **Ground** to link it to one or more brain nodes.

> **Expect:** an SVG graph with colour-coded nodes by type and labelled edges (auto-laid-out). The grounded task shows the linked node labels as chips. Deleting a node removes its edges (cascade).

6 Pipelines — run agents on a task headline test

a. Pick a small, self-contained task (for a first run, give it a trivial body like "*create a file hello.txt containing hi*" so it finishes fast and safely in the sandbox dir).

b. Open the task → **Run pipeline**. You jump to the **Pipelines** (🛠) view.

c. Watch the card: stage **developer** turns *running* (blue). Click the running stage chip to jump into the live agent's terminal in **Fleet** and watch it work.

d. When the developer finishes, the pipeline **auto-advances** to reviewer → integrator → documentor on its own.

> **Expect:** stages go queued → running → done (green) in order; on the last stage the run is marked **shipped** and the task moves to the **Shipped** column in Backlog automatically. Each stage's agent runs on the project's account (shown via the account dot).

> **This spawns real Claude agents** that consume your account and edit files in the project's working dir. Keep the first task tiny. To stop a run early: kill its tmux sessions — `tmux ls` then `tmux kill-session -t <name>` (names start with the stage role, e.g. `developer-…`).

7 Home — the dashboard reflects everything

a. Nav → **Home** (🏠).

> **Expect:** a KPI band with task counts per status (Ready / In Progress / Review / Shipped / Blocked), an **Active Pipelines** list (running stages), and **Recent Brain Nodes** with type chips. Clicking a KPI / pipeline / node jumps to the matching view.

8 Fleet & Orchestrator — conversational control (optional)

a. Nav → **Fleet** → the Orchestrator chat. Type a goal (e.g. "*plan a small change to the sandbox repo*").

> **Expect:** it decomposes the goal and spawns worker agents (visible in the Agents list with account dots); its permission/brake prompts surface in the right-hand panel.

9 Persistence & restart — nothing is lost

a. Stop the server (`Ctrl-C` in its terminal, or `pkill -f "uvicorn tui_pilot"`) and start it again with the same `TUI_PILOT_HOME=~/spade-qa`.

b. Reload `/ui/`.

> **Expect:** your accounts, projects, tasks, brain graph and pipelines are all still there (SQLite at `~/spade-qa/tui-pilot.db`). Any agent whose tmux session is still alive is **re-adopted**; dead ones show as exited. A pipeline that was mid-flight resumes advancing.

4 · Acceptance checklist

Tick each as you confirm it:

- ☐ **Accounts:** scan lists `~/.claude-*`; import registers one; set-default shows ⭐.
- ☐ **Projects:** create + working dir + account pool; current-project chip with account dots; round-robin preview rotates.
- ☐ **Agents:** 7 roles listed; edits to instructions/model persist; no account field.
- ☐ **Backlog:** tasks get sequential SPD ids; feature chip + priority dot; move across columns; header counts update.
- ☐ **Brain:** typed nodes render colour-coded; edges drawn + labelled; deleting a node cascades its edges.
- ☐ **Grounding:** a task links to brain nodes and shows them as chips.
- ☐ **Pipeline run:** developer stage spawns a real agent in the project dir.
- ☐ **Auto-advance:** stages progress developer→reviewer→integrator→documentor unattended.
- ☐ **Ship:** last stage marks the run shipped and moves the task to Shipped.
- ☐ **Home:** KPIs, active pipelines, recent nodes all reflect current state.
- ☐ **Persistence:** restart preserves all data; live agents reattach.

5 · Troubleshooting

Symptom	Cause / fix
Pipeline stage immediately **failed** / run **paused**	The project's account is *not logged in*. Go to Accounts → **Log in** on that account (complete the device login once), or set a default account that is authed.
`/ui/` won't load	Server not running, or wrong port. Check `~/spade-qa/server.log`; confirm `curl 127.0.0.1:8765/ui/` → 200.
`address already in use` on start	An old server is running: `pkill -f "uvicorn tui_pilot"` then relaunch.
Agent never starts / "tmux not found"	Install tmux (`brew install tmux`). Each agent needs a tmux session.
Empty Backlog/Brain after creating things	No **current project** selected — views are project-scoped. Pick one in the top-bar switcher.
Too many leftover agents	`tmux ls` to list, `tmux kill-session -t <name>` to stop one. The UI's **kill all** (Fleet) stops tracked sessions.

6 · Teardown

```
# stop the server
pkill -f "uvicorn tui_pilot"

# stop any agent tmux sessions
tmux ls 2>/dev/null | cut -d: -f1 | xargs -I{} tmux kill-session -t {}

# wipe the throwaway QA data (accounts/projects/tasks/brain/pipelines)
rm -rf ~/spade-qa
```

Full design & roadmap: `docs/spade-mvp-design` & `docs/spade-alignment.md` in the repo. Deferred (post-MVP): sprints, ADR-conflict gates, meetings/feedback/integration ingestion, conversational Brain (Ask), multi-provider accounts, cost tracking.